

MODEL DEVELOPMENT DOCUMENT

Daily Minimum Temperature Forecasting - Melbourne, Australia

Submitted by

VINEETH M

Applied DS, ML & AI
E & ICT Academy, IIT Guwahati
Capstone Project 2
September 2026

GitHub: <https://github.com/Vineeth-Muraleedharan/Daily-Minimum-Temperature-Forecasting->

Streamlit App: <https://daily-minimum-temperature-forecasting-by-vineeth.streamlit.app/>

1. Project Overview

1.1 Project Details

Attribute	Details
Project Title	Daily Minimum Temperature Forecasting - Melbourne, Australia
Project Type	Capstone Project 2 - E & ICT Academy, IIT Guwahati
Submitted By	Vineeth M
Domain	Time Series Forecasting / Environmental Data Analytics
Dataset	Daily Minimum Temperatures in Melbourne (Kaggle: paulbrabban/daily-minimum-temperatures-in-melbourne), 3,650 daily records, 1981-1990 (https://www.kaggle.com/datasets/paulbrabban/daily-minimum-temperatures-in-melbourne)
Tools Used	Python 3.12, VS Code, Jupyter Notebook, pandas, scikit-learn, XGBoost, statsmodels, TensorFlow/Keras
Deployment	Streamlit Cloud
GitHub	https://github.com/Vineeth-Muraleedharan

1.2 Problem Statement

Daily minimum temperature is a quantity that agriculture, energy demand planning, aviation, and public health services all depend on forecasting accurately, even one day ahead; frost warnings for crops, heating-load planning, and cold-weather health advisories all rely on it. This project develops a regression model to predict the next day's minimum temperature for Melbourne, Australia, using ten years (1981-1990) of daily historical readings from a single weather station.

Unlike a classification problem with a discrete target, this is a continuous-valued, univariate time series forecasting problem: the only information available is the series' own history (no humidity, pressure, wind, or satellite inputs are present in the dataset). The central technical challenge is therefore twofold: first, engineering features that expose the series' short-term persistence and annual seasonality to models that otherwise see i.i.d. tabular rows; and second, evaluating every model honestly, since it is easy for a time series model to appear skillful by leaking future information across a naive random train/test split, or by being scored on a multi-step forecast horizon that does not reflect how the model would actually be used.

1.3 Domain Relevance

The dataset itself is meteorological rather than clinical. What transfers is the methodology: engineering lag and rolling window features from a historical series, building calendar/cyclical encodings for recurring patterns, splitting, and validating chronologically to avoid lookahead leakage, and comparing classical statistical models against machine learning and deep learning approaches under an identical, fair evaluation protocol. These are the same techniques needed for healthcare operational forecasting problems; predicting daily patient volume in a radiotherapy department, equipment utilization patterns, or staffing demand from historical scheduling data - where the underlying time series structure (trend, weekly/seasonal cycles, autocorrelated noise) is analogous to the one studied here.

1.4 Project Objectives

- Perform exploratory data analysis to characterize the series' trend, seasonality, and noise structure
- Resolve data quality issues specific to the raw Kaggle export before any modeling begins
- Engineer calendar (Fourier), lag, and rolling-window features suitable for one-step-ahead forecasting
- Establish naive persistence and seasonal-naive baselines that any credible model must beat
- Train and compare Linear Regression, Ridge Regression, Random Forest, and tuned XGBoost models
- Benchmark against a classical SARIMAX (dynamic harmonic regression) model using proper walk-forward evaluation
- Build LSTM and GRU deep learning sequence models
- Compare all model families on RMSE, MAE, and R^2 under an identical chronological 80/20 test window
- Identify practical deployment trade-offs across model families rather than choosing on a single metric alone

2. Dataset Description

2.1 Dataset Overview

Attribute	Details
Source	Kaggle (paulbrabban/daily-minimum-temperatures-in-melbourne); underlying series originally published by the Australian Bureau of Meteorology (https://www.kaggle.com/datasets/paulbrabban/daily-minimum-temperatures-in-melbourne)
Total Records	3,650 rows as downloaded; 3,652 calendar days after cleaning (see Section 4.1)
Total Columns	2 raw columns (Date, Temp); 11 engineered features added for modeling (Section 4.4)
Target Variable	Temp - daily minimum temperature in °C
Time Span	1 Jan 1981 - 31 Dec 1990 (10 years)
Missing Values	2 calendar days absent from the raw file (recovered by reindexing and interpolation)
Duplicate Rows	None
Data Quality Flags	3 readings carry a Bureau-of-Meteorology "?" quality flag; the Kaggle export includes a trailing non-data description line

2.2 Feature Description (Raw)

Feature	Type	Range	Description
Date	Datetime	1981-01-01 to 1990-12-31	Calendar date of the reading
Temp	Continuous	0.0 - 26.3 °C	Daily minimum temperature recorded at a Melbourne weather station

3. Exploratory Data Analysis (EDA)

3.1 Time Series Overview and Trend Analysis

The full 10-year series shows a clear, repeating annual cycle with no discernible long-term multi-year trend: a seasonal decomposition (additive, period = 365) confirms the seasonal component's amplitude stays roughly constant across the decade, and the trend component is essentially flat. Across all 3,652 days, the mean minimum temperature is **11.18 °C** (std. dev. **4.07 °C**), ranging from **0.0 °C** to **26.3 °C**. This is the expected shape for a Southern-Hemisphere coastal city: warm summers (December–February) and cool winters (June–August), with genuine day-to-day noise superimposed on the seasonal signal.

3.2 Distribution Analysis

The distribution of Temp is close to symmetric: skewness = **0.171**, well within the range considered approximately normal ($|\text{skew}| < 0.5$), so no power or log transformation was applied before modeling. An IQR-based outlier check (bounds = $[Q1 - 1.5 \cdot \text{IQR}, Q3 + 1.5 \cdot \text{IQR}] = [-0.25 \text{ °C}, 22.55 \text{ °C}]$) flags **13 readings (0.36% of the series)** as statistical outliers, all of them unusually warm readings such as 26.3 °C on 1982-02-15 and 25.2 °C on 1982-01-20.

3.3 Seasonal Analysis - Monthly Patterns

Grouping by calendar month makes the annual cycle explicit and directly motivated the cyclical calendar features engineered in Section 4.4:

Month	Mean (°C)	Min (°C)	Max (°C)	Std. Dev.
January	15.03	8.5	25.2	2.87
February	15.37	9.2	26.3	2.74
March	14.57	7.4	22.4	3.19
April	12.09	5.7	21.8	3.08
May	9.87	2.1	16.5	2.79
June	7.28	0.0	13.0	2.64
July	6.69	0.0	13.0	2.63
August	7.89	1.7	14.3	2.33
September	8.98	3.0	19.2	2.80
October	10.31	4.7	18.4	2.69
November	12.48	5.7	24.3	2.96
December	13.86	8.2	23.9	2.47

Table 3.1 - Monthly summary statistics. July is the coldest month (mean 6.69 °C); February is the warmest (mean 15.37 °C), consistent with Melbourne's Southern-Hemisphere seasonal calendar.

3.4 Autocorrelation Analysis (ACF / PACF)

The autocorrelation function shows strong, gradually decaying correlation at short lags, the hallmark of an autoregressive process, confirming that recent days carry substantial predictive information about tomorrow's minimum. The partial autocorrelation function cuts off sharply after the first few lags, indicating most of the short-term dependence is captured within roughly a week of history. A secondary, weaker rise in autocorrelation is visible near lag 365, reflecting the annual recurrence of similar weather. These two patterns directly motivated the two families of lag features engineered in Section 4.4: short lags (1–14 days) for local persistence, and lag 365 for annual recurrence.

3.5 Correlation Analysis - Candidate Features vs. Target

Before finalizing the feature set, every candidate feature's correlation with the target (Temp) was computed on the full engineered dataset:

Feature	Correlation with Target	Interpretation
lag_1	0.772	Strongest predictor - yesterday's temperature is highly informative
roll_mean_7	0.730	7-day trailing average smooths noise while retaining local trend
roll_mean_30	0.710	Captures the slower seasonal drift
doy_cos	0.656	Encodes position in the annual cycle (winter vs. summer)
lag_2	0.623	Second-most-recent day
lag_3	0.578	Third-most-recent day
lag_7	0.570	One week prior
lag_365	0.535	Same calendar day one year prior; annual seasonality signal
lag_14	0.532	Two weeks prior
doy_sin	0.316	Second half of the annual cyclical encoding
roll_std_7	-0.025	Weak - retained as a local volatility indicator, not a level predictor

Table 3.2 - Pearson correlation of each candidate feature with the target. The dominance of lag_1 and the rolling means over lag_365 shows that recent local behaviour carries more signal here than the exact same day a year earlier.

4. Data Preprocessing

4.1 Handling Missing Values, Duplicates, and Kaggle Export Data Quality Issues

Three distinct data quality issues were identified in the raw Kaggle CSV, none of which are visible from a cursory inspection of the first few rows:

- **Trailing description row:** the export appends a non-data line after the last date ("Daily minimum temperatures in Melbourne, Australia, 1981-1990") containing an unquoted comma, which causes a plain `pd.read_csv()` call to fail with "Expected 2 fields, saw 3".
- **Quality-flagged readings:** 3 historical observations (1982-07-20, 1982-07-21, 1984-07-14) are recorded with a leading "?" (e.g. "?0.2") - a Bureau-of-Meteorology notation for a quality-flagged but genuine reading. A naive numeric conversion silently turns these into NaN and drops 3 real observations.
- **Missing calendar days:** 2 calendar days are absent from the raw file entirely.

A custom loader was written to resolve all three: malformed lines are skipped (`on_bad_lines='skip'`), columns are renamed positionally rather than by name (since the Kaggle export names the second column after the dataset title, not "Temp"), the "?" prefix is stripped before numeric conversion, and the series is reindexed onto a complete daily calendar with the 2 missing days filled by linear interpolation. This recovers the full 3,650 genuine readings and yields a continuous 3,652-day series with zero missing values and zero duplicates.

4.2 Skewness Analysis

Skewness of the target variable was computed to determine whether a transformation was warranted before modeling (skew = 0.171, approximately symmetric - see Section 3.2). No transformation was applied.

4.3 Outlier Analysis - Why Extremes Were Preserved

Unlike sensor-generated industrial data, where an extreme reading often indicates a faulty instrument, the 13 statistical outliers identified in Section 3.2 are genuine heatwave events: real, physically meaningful weather. Removing or winsorizing them would discard exactly the information a forecasting model most needs to learn about rare-but-real extremes and would understate the true variability of the series. All 13 readings were therefore retained without treatment.

4.4 Feature Engineering

Eleven features were engineered in three groups, each directly motivated by the EDA findings in Section 3:

Cyclical Calendar (Fourier) Features - `doy_sin`, `doy_cos`

$\text{doy_sin} = \sin(2\pi \cdot \text{day_of_year} / 365.25)$, $\text{doy_cos} = \cos(2\pi \cdot \text{day_of_year} / 365.25)$. A raw day-of-year integer (1–365) would incorrectly treat December 31 (day 365) and January 1 (day 1) as maximally distant, when they are adjacent in the annual cycle. The sine/cosine pair encodes calendar position on a continuous circle, so the model correctly sees late December and early January as similar.

Lag Features - `lag_1`, `lag_2`, `lag_3`, `lag_7`, `lag_14`, `lag_365`

Six lagged values of Temp, chosen directly from the ACF/PACF analysis in Section 3.4: short lags (1–14 days) capture local persistence, and `lag_365` captures the annual recurrence identified as a secondary autocorrelation peak.

Rolling-Window Statistics - roll_mean_7, roll_std_7, roll_mean_30

7-day and 30-day trailing mean and standard deviation, computed on Temp.shift(1) so the target day's own value is never included — this is essential to prevent lookahead leakage. The 7-day mean smooths short-term noise while tracking local trend; the 30-day mean tracks the slower seasonal drift; the 7-day standard deviation captures local volatility.

Justification: the correlation table in Section 3.5 shows every engineered feature except roll_std_7 correlates at 0.3 or higher with the target, with roll_mean_7 (0.730) and roll_mean_30 (0.710) competitive with the strongest raw lag (lag_1, 0.772), confirming the engineering choices add real predictive value rather than redundant noise.

4.5 Categorical Encoding

Not applicable. The raw dataset contains a single numeric measurement column (Temp) and a Date column, which is decomposed entirely into the numeric cyclical and lag features above rather than encoded as a category.

4.6 Train-Test Split - Chronological 80/20

The dataset was split 80:20 **chronologically**, not randomly: the first 80% of days (in calendar order) form the training set, and the last 20% form the test set. A random shuffle-based split, standard for i.i.d. tabular data, is inappropriate here: it would let the model train on days that occur after the days it is tested on, silently leaking future information (e.g. training on tomorrow's actual lag_1 while testing on today) and producing an optimistic, unrealistic accuracy estimate.

After dropping the first year of data (required so that lag_365 is defined for every remaining row), the split yields **2,629 training days** and **658 test days**, covering the test window 1989-03-14 to 1990-12-31. The classical (SARIMAX) and deep learning (LSTM/GRU) models in Sections 6-8 do not need to drop the first year, since they do not depend on lag_365 as an input feature - but they are deliberately re-aligned to forecast the identical 658-day test window, so that RMSE, MAE, and R^2 are directly comparable across every model family.

4.7 Feature Scaling

Model Family	Scaling Applied	Justification
Linear Regression, Ridge	None	Coefficients remain interpretable in original lag/°C units; regularization strength in Ridge was not tuned per-scale
Random Forest, XGBoost	None	Tree-based splits are invariant to monotonic feature scaling
SARIMAX	None	Exogenous Fourier terms are already bounded in [-1, 1]; the AR/MA structure operates on the raw °C series
LSTM, GRU	StandardScaler (fit on train only)	Gradient-descent training converges more reliably with zero-mean, unit-variance inputs; scaler fit only on the training portion to avoid leakage, then applied to the full series

5. Forecasting Protocol and Baseline Models

5.1 Why Naive Baselines Matter for Time Series

Unlike classification, where a majority-class baseline is the standard reference, a time series forecast can look deceptively skillful while doing nothing more than learning "tomorrow $\approx$ today". Before any machine learning model is judged as good, it must clear a naive persistence baseline by a meaningful margin - otherwise the added modeling complexity is not earning its keep.

5.2 Baselines Evaluated

- **Persistence:** predict tomorrow's minimum as today's actual value (lag_1).
- **Seasonal-naive:** predict tomorrow's minimum as the value recorded on the same calendar day one year earlier (lag_365).

5.3 Baseline Results

Model	RMSE (°C)	MAE (°C)	R ²
Naive (persistence)	2.513	1.974	0.579
Seasonal naive (1 yr)	3.706	2.926	0.085

Table 5.1 - Baseline performance on the 658-day test window. Persistence is the far stronger baseline, consistent with lag_1's higher correlation with the target (0.772) than lag_365's (0.535): any credible model must beat an RMSE of 2.513 °C by a real margin, not merely match it.

6. Model Development

6.1 Model Selection Rationale

Seven models across three methodological families were trained and compared:

Linear Regression

A linear model over the eleven engineered features, included as the interpretable baseline against which every other model is judged. Its coefficients are directly readable in physical units (°C per unit change in each lag/rolling feature).

Ridge Regression

L2-regularized linear regression, included to check whether the modest multicollinearity among lag features (e.g. lag_1 through lag_14 are all correlated with each other) meaningfully changes the fit relative to plain Linear Regression.

Random Forest

A bagged ensemble of decision trees, included to test whether non-linear interactions between calendar position and recent lags (which a linear model cannot represent) add predictive value.

XGBoost (tuned)

A gradient-boosted tree ensemble, tuned via RandomizedSearchCV. Consistently a strong performer on structured/tabular data in general, so its performance here is a useful test of whether this specific problem rewards that additional model capacity.

SARIMAX + Fourier (Dynamic Harmonic Regression)

A classical time series benchmark. A true seasonal ARIMA with an annual period ($s = 365$) is computationally impractical in statsmodels' state-space implementation, so the annual cycle is instead supplied as exogenous Fourier sine/cosine regressors (3 harmonic pairs), with a small ARIMA modeling the residual short-term autocorrelation on top: the standard "dynamic harmonic regression" approach for daily data with annual seasonality (Hyndman & Athanasopoulos, 2021).

LSTM

A recurrent neural network reading a rolling 30-day window of [scaled temperature, day-of-year sine, day-of-year cosine] to predict the next day's scaled temperature. Included to test whether a model that learns its own temporal representation, rather than relying on hand-engineered lag features, can improve on the ML models above.

GRU

The same windowed setup as the LSTM, with a GRU layer in place of an LSTM layer (fewer gates and parameters, often comparable accuracy with faster training on series of this size).

6.2 Evaluation Metrics Selection

Metric	Formula	Why Used
RMSE	$\sqrt{\text{mean squared error}}$	Primary metric - same units ($^{\circ}\text{C}$) as the target; penalizes large errors more heavily than small ones
MAE	mean absolute error	Secondary - a more directly interpretable "typical error in $^{\circ}\text{C}$ ", less sensitive to a handful of large misses
R^2	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	Proportion of variance explained - allows comparison against the naive baselines on a normalized scale

6.3 Training Configuration

- **Linear Regression, Ridge, Random Forest, XGBoost:** trained on the 3,287-row feature-engineered dataset (after dropping the first year for lag_365), chronologically split 2,629 train / 658 test.
- **XGBoost hyperparameter search:** RandomizedSearchCV, 25 iterations, TimeSeriesSplit as the cross-validation scheme (so every fold still trains only on the past), scoring = negative RMSE.
- **SARIMAX:** trained on the full 3,652-day series (no need to drop the first year), then evaluated on the identical 658-day test window using proper one-step-ahead walk-forward forecasting - forecast one day, feed in the true observed value, forecast the next.
- **LSTM / GRU:** 30-day lookback windows; StandardScaler fit on the training portion only; a chronological validation split (the last 10% of training sequences) used for early stopping (patience = 8 epochs).

- **Reproducibility:** random_state / random seeds fixed at 42 throughout (NumPy, scikit-learn, XGBoost, TensorFlow).

7. Model Results and Comparison

7.1 Performance Summary

Model	RMSE (°C)	MAE (°C)	R ²
LSTM	2.145	1.692	0.694
GRU	2.148	1.696	0.693
Blend (Linear Regression + XGBoost)	2.149	1.691	0.693
Linear Regression	2.155	1.699	0.691
Ridge Regression	2.155	1.699	0.691
SARIMAX + Fourier (walk-forward)	2.163	1.714	0.688
Random Forest	2.163	1.698	0.688
XGBoost (tuned)	2.180	1.715	0.683
Naive (persistence)	2.513	1.974	0.579
Seasonal naive (1 yr)	3.706	2.926	0.085

Table 7.1 - All models evaluated on the identical 658-day test window (1989-03-14 to 1990-12-31). Highlighted rows would mark the best-performing (LSTM) and the recommended deployment model (Linear Regression): see Section 8.1.

7.2 Key Findings

Finding 1: Every real model converges to a tight RMSE band

All seven trained models land within **2.145–2.180 °C RMSE** - a spread of just 0.035 °C - regardless of whether the model is linear, tree-based, gradient-boosted, classical statistical, or a recurrent neural network. This convergence across five methodologically distinct families is itself the central finding: with only one station's own past minimum temperatures as input, roughly 2.1-2.2 °C of day-to-day variation is not predictable from this data, and no additional modeling sophistication removes it.

Finding 2: Evaluation protocol matters as much as model choice

An early version of the SARIMAX model, evaluated with a single static 658-day-ahead forecast produced RMSE 2.55–2.62 °C - noticeably worse than the ML models. This was misleading: the ML models effectively receive tomorrow's true lag_1 at every test step, while a static multi-step SARIMAX forecast does not get to see any of the 658 intervening true values. Re-evaluating SARIMAX under a proper one-step-ahead walk-forward protocol: forecast one day, incorporate the true observed value, forecast the next, brought it to RMSE 2.163 °C, on par with the ML models. The lesson: comparing models fairly in a forecasting context requires matching the evaluation protocol to how each model would be used, not just running whichever method a library exposes by default.

Finding 3: The lowest-AIC SARIMA order did not win on forecast skill

A small AIC-based grid search over favored orders such as (1,1,2) and (3,1,1), but under the walk-forward evaluation protocol these performed statistically indistinguishably from the simpler order (2,0,2), which was ultimately selected (RMSE 2.163 °C vs. 2.164-2.165 °C for the AIC-favored alternatives). AIC measures in-sample fit; walk-forward RMSE measures actual forecasting skill on unseen data: the two criteria did not fully agree here, a reminder that automated order selection should be checked against genuine out-of-sample performance rather than trusted alone.

Finding 4: Additional model capacity buys almost nothing

Linear Regression (RMSE 2.155 °C) is statistically indistinguishable from the LSTM (2.145 °C) at this sample size ($n = 658$). Once the annual cycle and short-term persistence are made explicit as engineered features, the relationship between them and tomorrow's minimum temperature is close to linear - so the extra representational capacity of tree ensembles, gradient boosting, and recurrent neural networks does not translate into a materially better forecast for this specific univariate series.

7.3 Hyperparameter Tuning

XGBoost

RandomizedSearchCV (25 iterations, TimeSeriesSplit with 5 folds, scoring = negative RMSE) searched over `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, and `min_child_weight`. The selected configuration — `max_depth = 2`, `learning_rate = 0.01`, `n_estimators = 300`, `subsample = 0.6`, `colsample_bytree = 1.0`, `min_child_weight = 3` - favors a shallow, conservative ensemble, consistent with Finding 4: the search itself found that a near-linear model generalizes best.

SARIMAX Order Selection

A grid over $p \in \{1,2,3\}$, $d \in \{0,1\}$, $q \in \{0,1,2\}$ was ranked by AIC on the training data; the top three AIC candidates plus order (2,0,2) were then confirmed under the walk-forward protocol described in Finding 2. Final selection: order (2,0,2) with $K = 3$ Fourier harmonic pairs (6 exogenous regressors).

8. Final Model Selection and Justification

8.1 Selected Model

Because the eight real models are separated by only 0.035 °C RMSE, no single model is a clear winner on accuracy alone. Two distinct recommendations are made instead, each optimizing for a different criterion:

Criterion	Recommendation	Justification
Best performing (by RMSE)	LSTM	Lowest test RMSE (2.145 °C) and MAE (1.692 °C) of all models evaluated
Recommended for deployment	Linear Regression (or the Linear + XGBoost blend)	Statistically indistinguishable accuracy from the LSTM, at a fraction of the infrastructure cost - see Section 8.3

8.2 Operational Interpretation

On the 658-day test set, the recommended Linear Regression model's forecasts are off by about **1.70 °C on average (MAE)**, and about **2.15 °C in a root-mean-square sense (RMSE)**. Relative to the series' own standard deviation of 4.07 °C, the residual error is roughly half of one standard deviation: a meaningful, non-trivial improvement over simply guessing the historical mean. Relative to the naive persistence baseline (RMSE 2.513 °C), every trained model reduces error by roughly **14-15%**. In practical terms: a forecast of "9 °C tomorrow" should be read as "most likely between about 7 °C and 11 °C", which is precise enough to be useful for frost warnings, heating-load planning, and similar applications, while making clear that day-to-day weather retains genuine unpredictability that no model in this study removes.

8.3 Model Choice Trade-offs

Consideration	Linear Regression / Blend	LSTM / GRU
Infrastructure	None beyond scikit-learn	TensorFlow required; GPU optional but speeds training
Training time	Under 1 second	14–50 seconds (CPU, this dataset size)
Interpretability	Coefficients readable in °C per feature unit	Effectively a black box without additional tooling
Retraining cadence	Trivial to retrain daily/weekly	Requires managing a training pipeline, checkpoints, and a scaler
Accuracy gains over Linear Regression	(reference)	0.01 °C RMSE - not distinguishable from noise at n = 658

Given a 0.01 °C RMSE difference is well within the range attributable to random variation at this sample size, the recommendation is to deploy Linear Regression (or the Linear + XGBoost blend, which ties the LSTM most closely) unless a future iteration adds richer inputs - multivariate weather data or a longer historical record: where the LSTM's greater representational capacity would have more signal to exploit.

8.4 Feature Importance Analysis

Feature importance from the tuned XGBoost model:

Rank	Feature	Importance	Type
1	roll_mean_7	0.4616	Engineered - rolling
2	roll_mean_30	0.1965	Engineered - rolling
3	lag_1	0.1402	Engineered - lag
4	doy_cos	0.1152	Engineered - calendar
5	doy_sin	0.0173	Engineered - calendar
6	lag_7	0.0139	Engineered - lag
7	lag_365	0.0129	Engineered - lag
8	roll_std_7	0.0126	Engineered - rolling
9	lag_2	0.0101	Engineered - lag
10	lag_14	0.0101	Engineered - lag
11	lag_3	0.0098	Engineered - lag

Table 8.1 - The 7-day rolling mean alone accounts for 46% of the model's total importance, confirming that recent local trend, rather than any single day's reading or the precise seasonal position- carries most of the predictive signal, and validating the rolling-window feature engineering decision in Section 4.4.

9. Model Deployment

9.1 Deployment Status

Unlike Capstone 1, a live Streamlit deployment has not yet been built for this project. The remainder of this section describes the recommended architecture, consistent with the Capstone 1 deployment pattern, for a future iteration of this work.

9.2 Proposed Application Architecture

- **Model loading:** the recommended Linear Regression model (or the Linear + XGBoost blend) loaded from a serialized pickle file at startup using `@st.cache_resource` for efficiency
- **Feature computation:** given a target date and the trailing 30-365 days of actual readings, the app automatically computes the required lag, rolling-window, and Fourier calendar features
- **Prediction:** computed features passed directly to the loaded regression model.
- **Output:** a point forecast in °C, accompanied by an uncertainty band derived from the residual standard deviation observed on the test set

9.3 Proposed Application Features

- Date picker with a rolling display of the last N days' actual readings
- Next-day forecast with a $\pm$ residual-based uncertainty band
- Toggle to compare the recommended model's forecast against the naive persistence baseline
- Hosted on Streamlit Cloud.

9.4 Deployment Files

File	Purpose	Status
capstone2_temperature_forecast.ipynb	Complete analysis notebook	Complete
Temperature.csv	Original dataset	Complete
requirements.txt	Python dependencies	Complete
app.py	Streamlit application code	Complete
model.pkl	Serialized Linear Regression model	Complete
scaler.pkl	Serialized StandardScaler (for an LSTM/GRU variant, if deployed instead)	Complete

10. Conclusions and Future Work

10.1 Summary of Findings

- A robust, quality-checked 3,652-day series was recovered from a raw Kaggle export containing three distinct, non-obvious data quality issues
- Eleven engineered features (cyclical calendar, six lags, three rolling-window statistics), all justified against EDA-derived correlation and autocorrelation evidence, drive every model in this study
- Seven models spanning three methodological families - linear/tree ML, classical statistical (SARIMAX), and deep learning (LSTM/GRU) - converge to a tight RMSE band of 2.145-2.180 °C, evidence of an irreducible noise floor given single-station inputs
- Fair, protocol-matched evaluation (chronological splitting, walk-forward SARIMAX forecasting) materially changed the classical model's apparent performance, underscoring that evaluation methodology matters as much as model choice
- Linear Regression matches the best deep learning model's accuracy at a fraction of the infrastructure cost, and is the recommended deployment choice

10.2 Limitations

- Single-station, univariate input only - no humidity, pressure, wind, or satellite data
- A 10-year historical window may not fully characterize rare, multi-decade extreme events
- Evaluation covers one-step-ahead forecasting only; multi-day-ahead forecast skill was not assessed
- LSTM/GRU were trained with a single random seed and a single architecture; a broader architecture or hyperparameter search was out of scope for this capstone

10.3 Future Work

- Add exogenous meteorological features (humidity, pressure, an ENSO index) to test whether the ~2.1 °C ceiling can be lowered
- Extend to multi-step forecasting (a 7-day horizon) via recursive or direct multi-output strategies
- Explore attention-based or Temporal Fusion Transformer architectures to test whether more model capacity ever breaks through the observed ceiling
- Apply the same forecasting methodology - lag/rolling feature engineering, chronological validation, walk-forward evaluation - to a healthcare operations dataset, such as radiotherapy department patient volume or equipment utilization forecasting

11. References

- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.
- Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP 2014*.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD 2016*.
- Seabold, S., & Perktold, J. (2010). statsmodels: Econometric and Statistical Modeling with Python. *Proceedings of the 9th Python in Science Conference*.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.
- Abadi, M., et al. (2016). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems.
- Dataset: Kaggle - paulbrabban/daily-minimum-temperatures-in-melbourne (originally published by the Australian Bureau of Meteorology) (<https://www.kaggle.com/datasets/paulbrabban/daily-minimum-temperatures-in-melbourne>).

VINEETH M

Applied DS, ML & AI

E & ICT Academy, IIT Guwahati

Email: vinuvineeth616@gmail.com

Mob: +91 7560946570

Project Milestones

Milestone	Status & Description
Define a problem	Complete - Identified daily minimum temperature forecasting as a univariate time series regression problem
Understanding the business problem	Complete - Reviewed practical relevance (agriculture, energy, public health); defined success metrics (RMSE, MAE, R^2) and a fair walk-forward evaluation protocol
Get the Data	Complete - Obtained the Daily Minimum Temperatures in Melbourne dataset (3,650 records, 1981-1990) from Kaggle (https://www.kaggle.com/datasets/paulbrabban/daily-minimum-temperatures-in-melbourne)
Explore and pre-process data	Complete - EDA performed (trend, seasonality, ACF/PACF, correlation); three data quality issues resolved; outliers reviewed and retained
Choosing the Python platform	Complete - Python 3.12 selected with VS Code and Jupyter Notebook (conda environment); pandas, scikit-learn, XGBoost, statsmodels, TensorFlow/Keras
Create Features	Complete - Eleven engineered features: cyclical calendar (Fourier) terms, six lag features, three rolling-window statistics
EDA	Complete - Univariate, seasonal, and autocorrelation analysis; correlation of all candidates features against the target computed
Create Model	Complete - Seven models trained: Linear Regression, Ridge, Random Forest, XGBoost, SARIMAX (dynamic harmonic regression), LSTM, GRU
Model Evaluation	Complete - Models compared on RMSE, MAE, R^2 under an identical chronological test window; XGBoost tuned via RandomizedSearchCV + TimeSeriesSplit; SARIMA order selected via AIC + walk-forward validation
Report Writing	Complete - Model Development Document (MDD) prepared with full justification for all decisions
Project submission	Complete - Jupyter notebook, MDD and Streamlit deployment completed.

GitHub: <https://github.com/Vineeth-Muraleedharan/Daily-Minimum-Temperature-Forecasting->Streamlit App: <https://daily-minimum-temperature-forecasting-by-vineeth.streamlit.app/>